

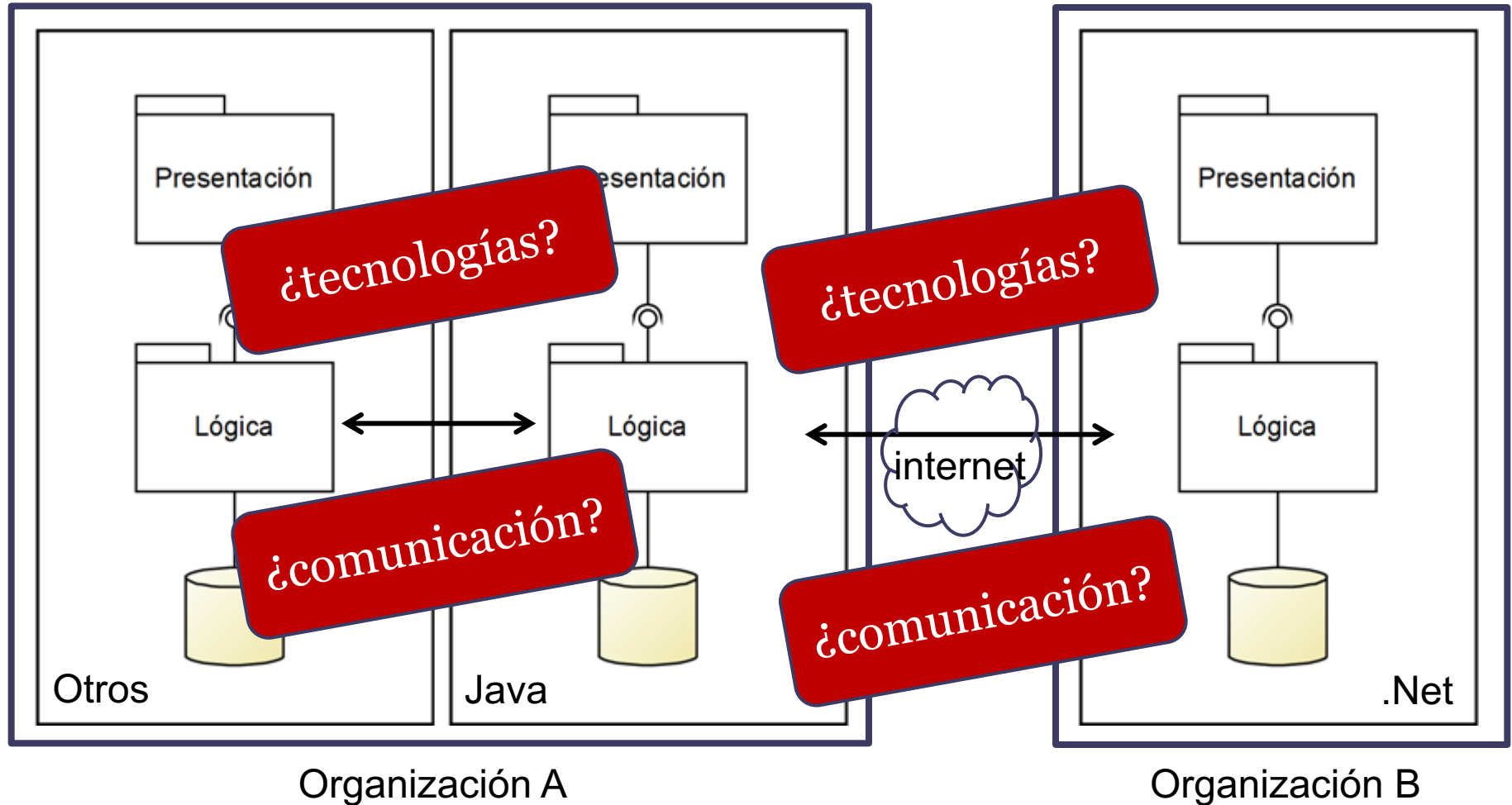
Introducción a Web Services

Taller de Programación 2023

tprog@fing.edu.uy



Introducción



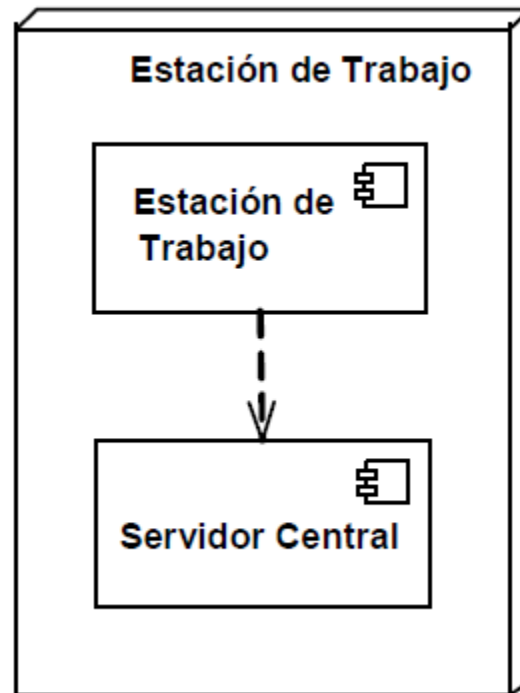
Introducción

- Sistemas distribuidos
 - procesamiento de la información está distribuido en dos o más computadoras en red
- mayor complejidad requieren considerar elementos específicos
 - middleware, escalabilidad, apertura, tolerancia a fallas, concurrencia, etc.
- en general los sistemas de gran escala hoy en día son sistemas distribuidos

[Sommerville, 2016]

Introducción

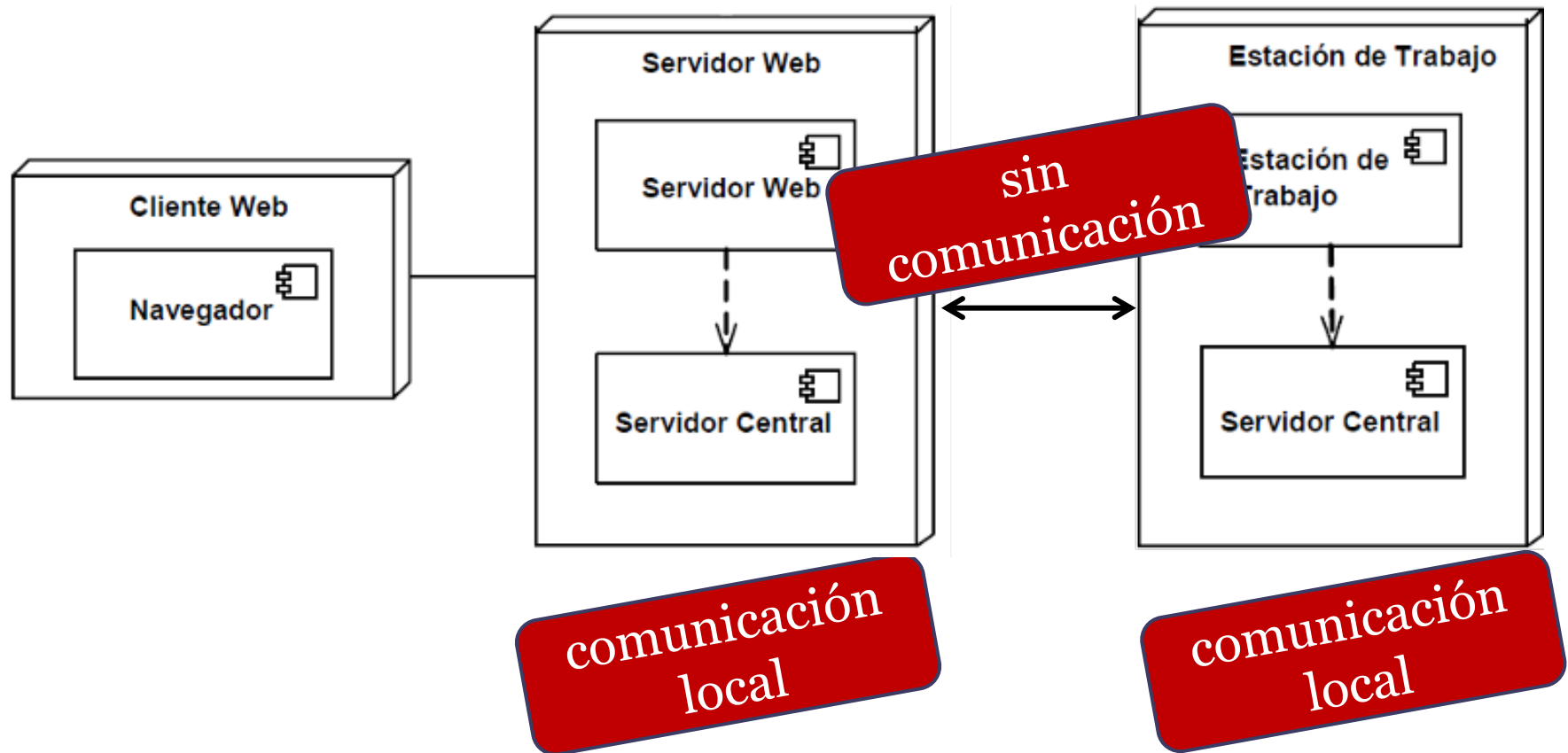
- Diagrama de distribución Tprog Tarea 1



**comunicación
local**

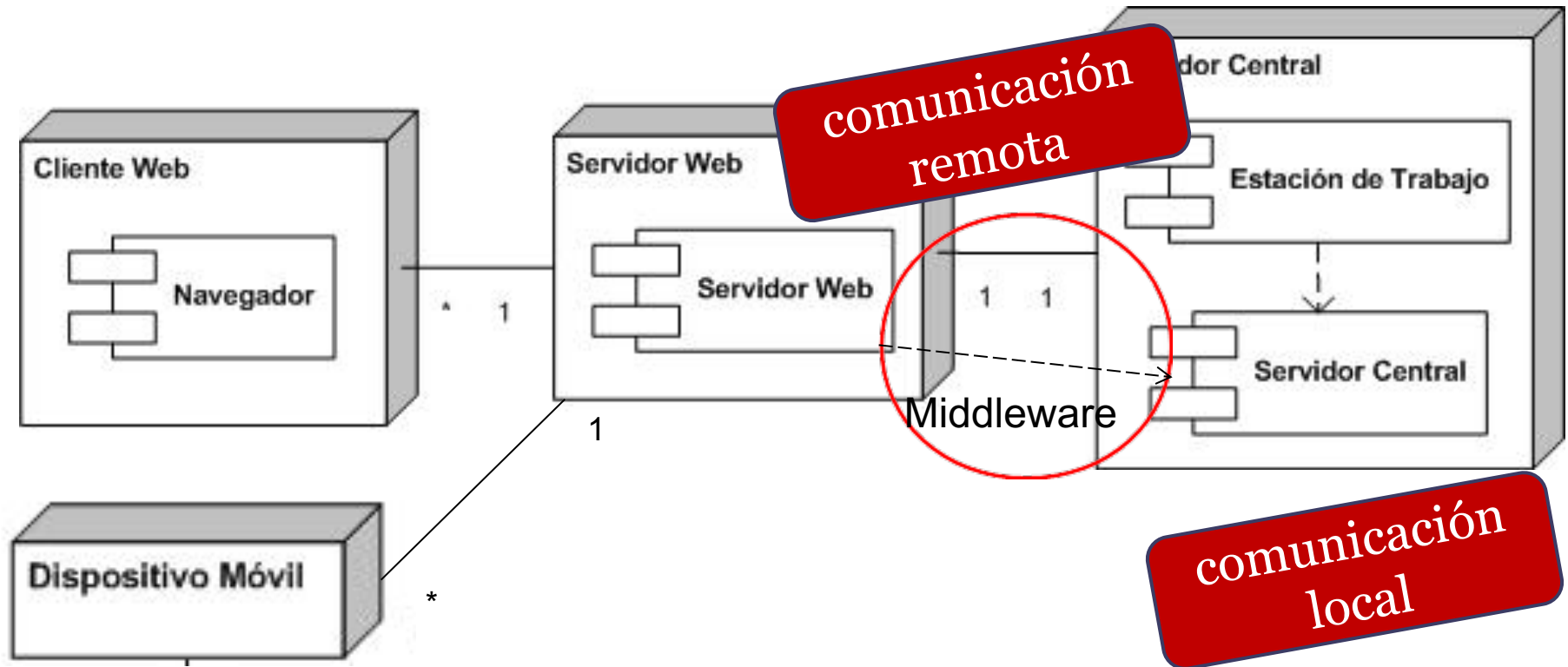
Introducción

- Diagrama de distribución Tprog Tarea 2



Introducción

- Diagrama de distribución Tprog Tarea 3

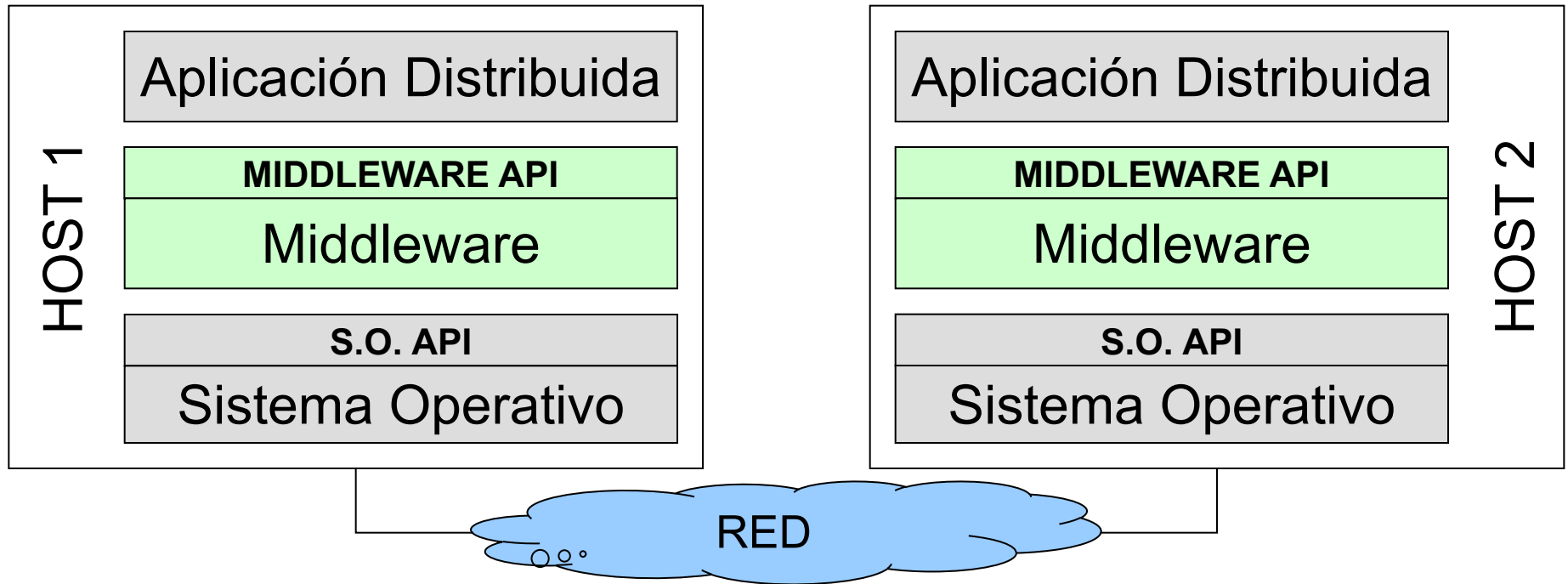


Introducción

- ¿Qué es el middleware ?
 - es el software que permite gestionar partes de un sistema distribuido asegurando que se pueden comunicar e intercambiar datos
 - Permite las interacciones a nivel de aplicación entre distintos componentes en un ambiente distribuido
 - Posicionado entre una aplicación y un sistema de menor nivel (S.O., DBMS, Servicio Red).
 - Sin middleware se complica el desarrollo de sistemas distribuidos
 - Programación de módulos de bajo nivel cada vez

Introducción

- ¿Qué es el middleware ?



Introducción

- Tipos de middleware
 - Sockets, ODBC/JDBC (Bases de datos)
 - Remote Procedure Call (RPC, RMI, CORBA, DCOM)
 - Invocación a procedimientos remotos como locales.
 - Message Oriented Middleware (MOM)
 - Mensajería (RabbitMQ)
 - AMQP (Advanced Message Queuing Protocol)
 - Web Services (SOAP, REST).
 - SOAP (Simple Object Access Protocol)
 - REST (Representational state transfer)

Web Services

- Definidos por la W3C (World Wide Web Consortium)
 - Estándares sobre la web (XML, WSDL, SOAP)
 - Primeros protocolos definidos a fines del siglo XX
- Motivado por la necesidad de
 - comunicar sistemas implementados en distintas tecnologías
 - habilitar comunicaciones vía Internet más fácilmente
- Proveen una manera estándar
 - de interoperar entre diferentes aplicaciones de software ejecutando en distintas plataformas

Web Services

- Tipos

- SOAP

- Orientado a operaciones
 - Protocolo sobre HTTP - librerías abstraen la complejidad del uso real de SOAP
 - Precisa la utilización de XML

- REST

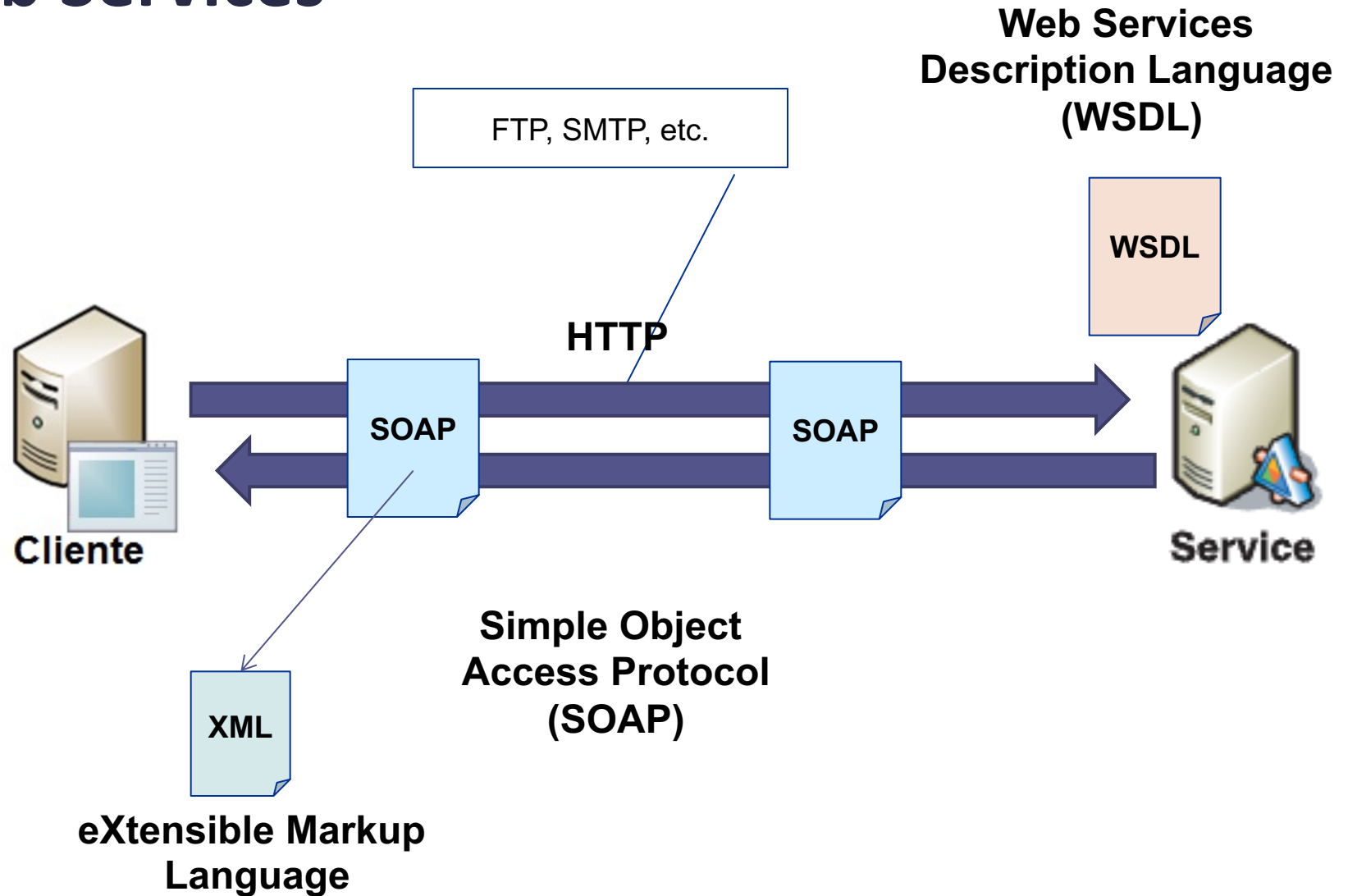
- Orientado a entidades como recursos sobre la web
 - Aprovecha métodos HTTP (GET, POST, PUT, DELETE)
 - No tiene el overhead de SOAP
 - Puede utilizar XML, Json, etc

Web Services

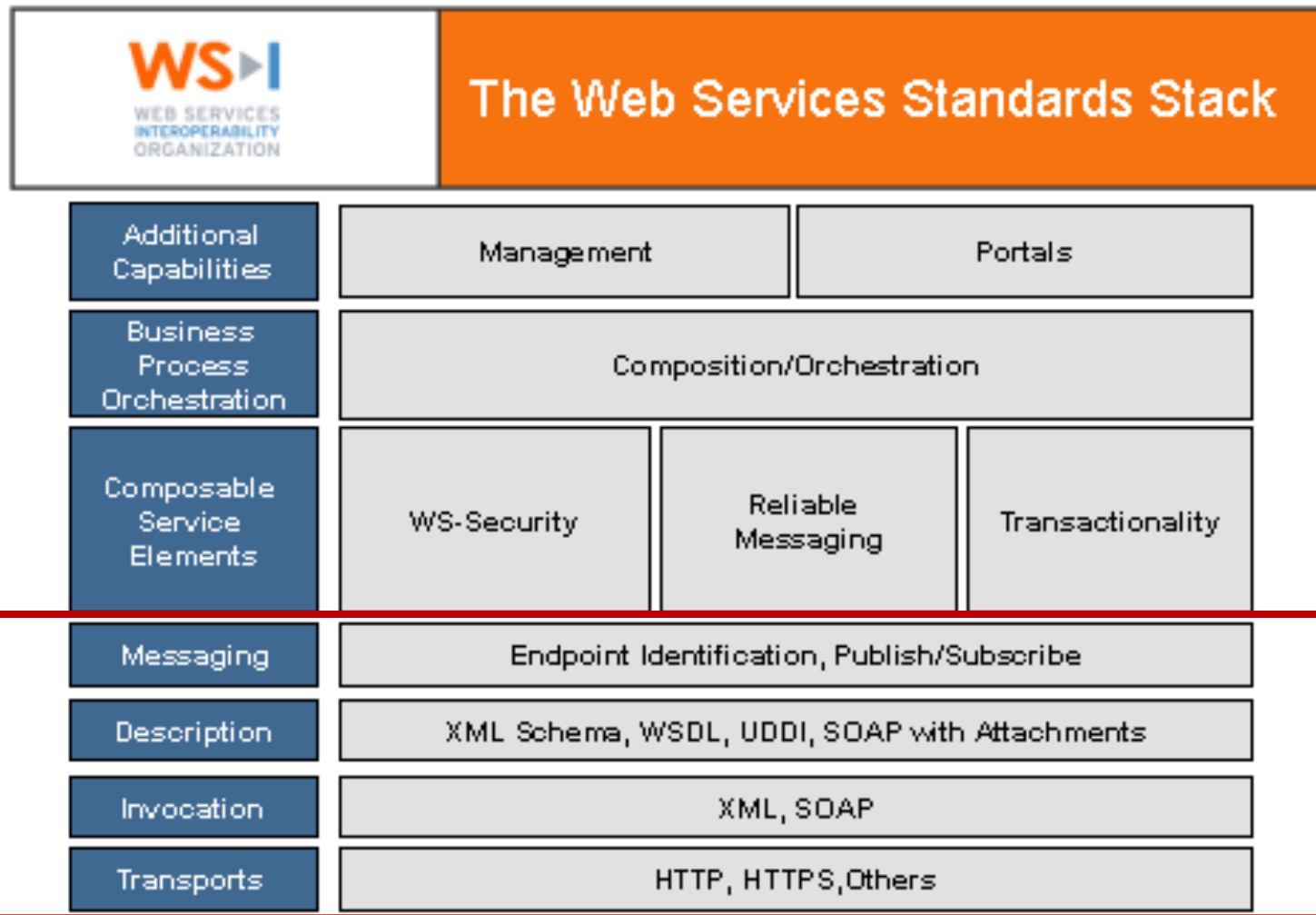
- “A Web Service is a **software system** designed to support **interoperable machine-to-machine interaction over a network**. It has an interface described in a machine-processable format (specifically **WSDL**). Other systems interact with the Web service in a manner prescribed by it’s description using **SOAP messages**, typically conveyed using **HTTP** with an **XML** serialization in conjunction with other Web-related standards.”

World Wide Web Consortium (W3C), 2006

Web Services

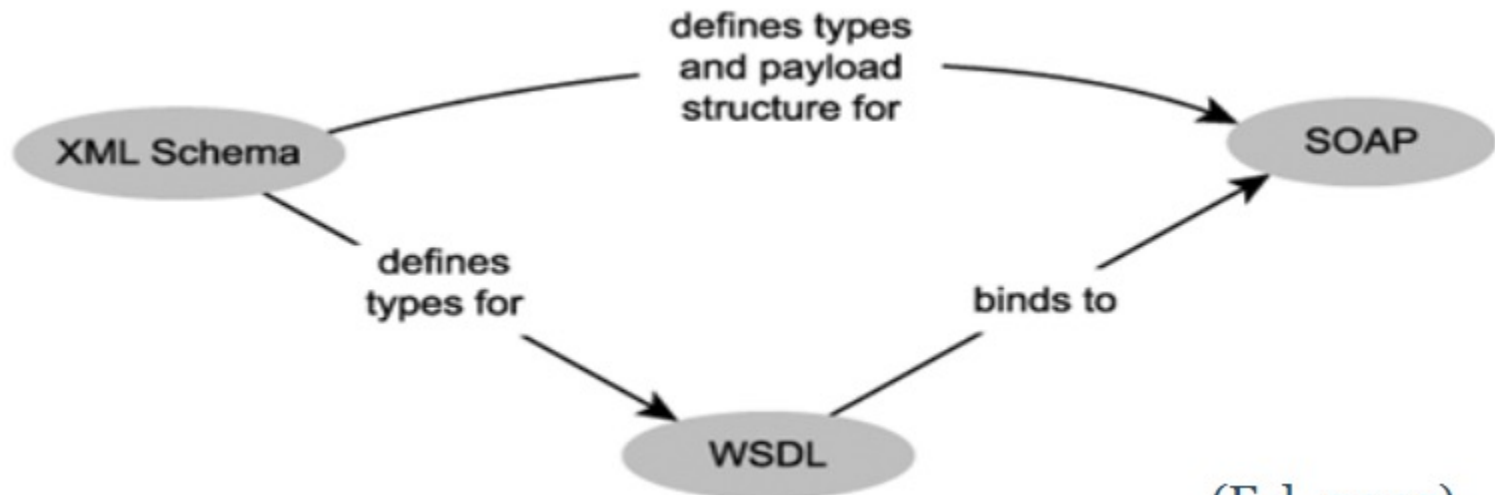


Web Services



Web Services

- Elementos en la definición de Servicios Web SOAP
 - XML Schema (XSD) para definir los tipos en los mensajes
 - WS Description Language (WSDL) para especificar los WS
 - Simple Object Access Protocol (SOAP) para comunicación



(Erl, 2005)

Web Services

- XML Schema (XSD) define la estructura de un documento XML (eXtensible Markup Language)
 - Tipos, atributos y sus relaciones válidas

XML Schema

```
5. <xs:element name="Customer">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="Dob" type="xs:date" />
            <xs:element name="Address" type="xs:string" />
        </xs:sequence>
    </xs:complexType>
</xs:element>

10. <xs:element name="Supplier">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="Phone" type="xs:integer"/>
            <xs:element name="Address" type="xs:string"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>
```

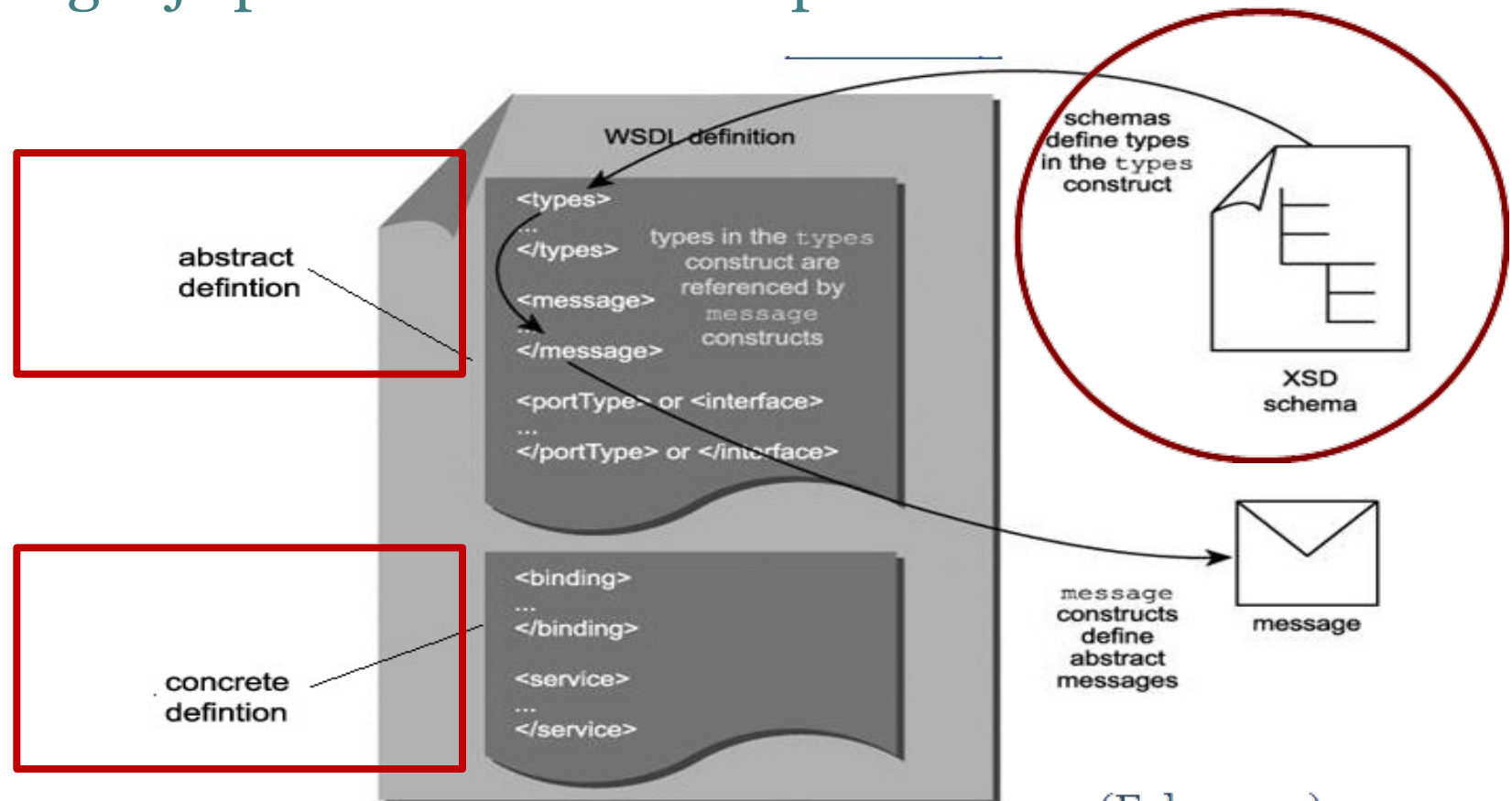
XML válido

```
5. <Customer>
    <Dob> 2000-01-12T12:13:14Z </Dob>
    <Address>
        34 thingy street, someplace, sometown, w1w8uu
    </Address>
</Customer>

10. <Supplier>
    <Phone>0123987654</Phone>
    <Address>
        22 whatever place, someplace, sometown, ss1 6gy
    </Address>
</Supplier>
```


Web Services

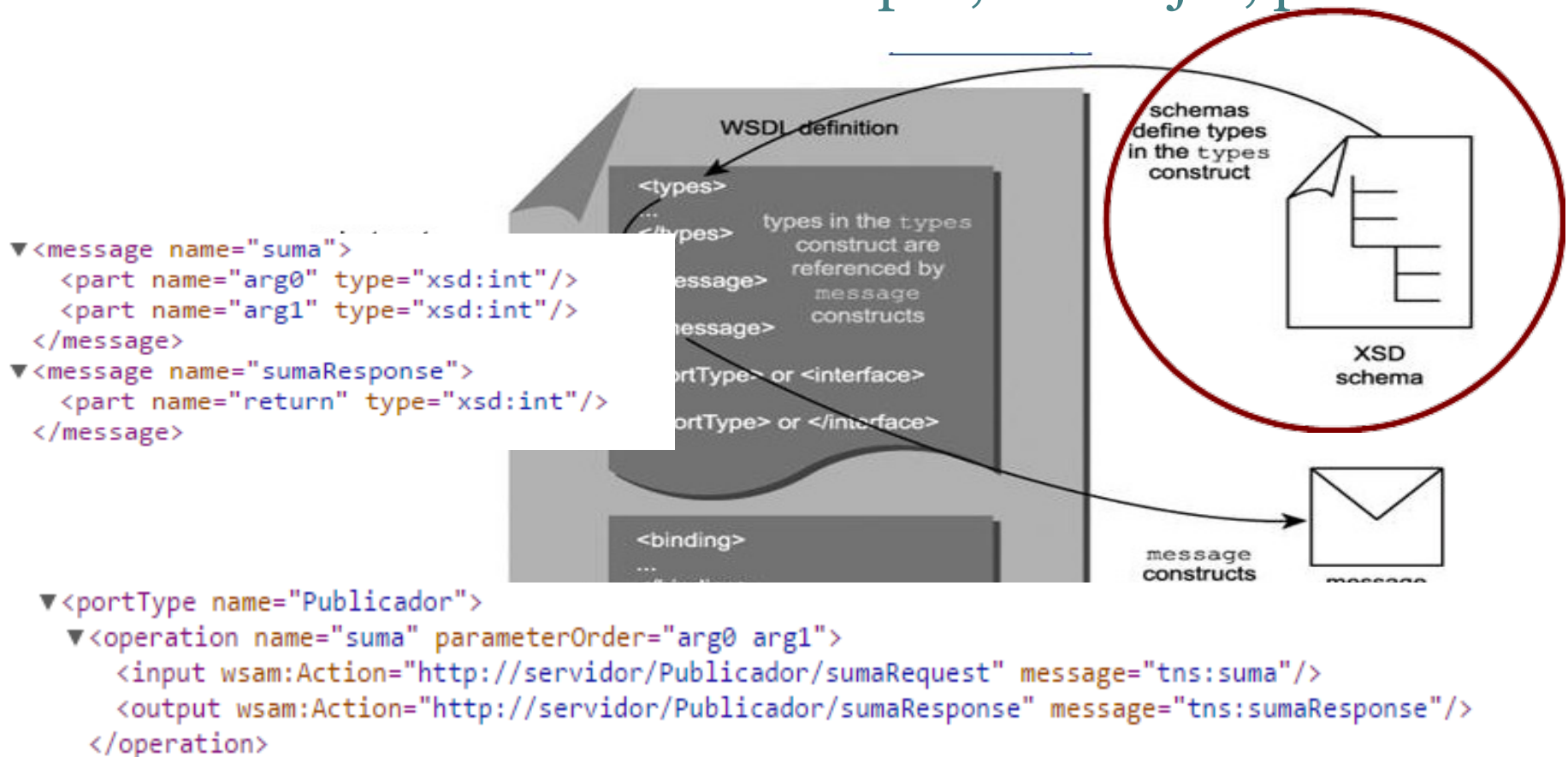
- Web Services Description Language (WSDL)
 - Lenguaje para definición completa de WS



(Erl, 2005)

Web Services

- Web Services Description Language (WSDL)
 - Definición abstracta define tipos, mensajes, puertos

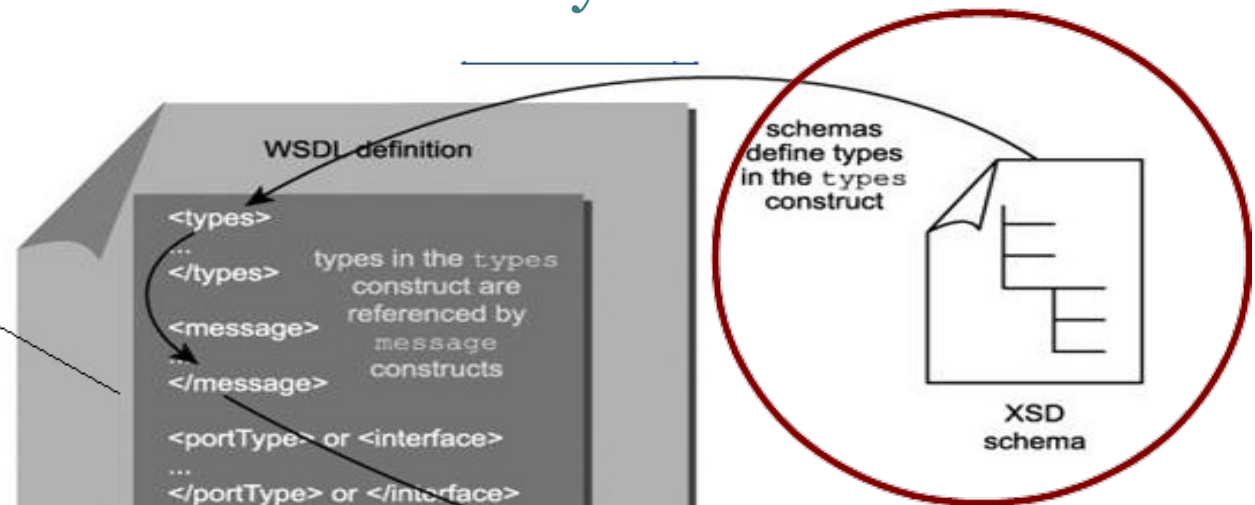


Web Services

- Web Services Description Language (WSDL)
 - En la definición concreta se incluyen detalles sobre

- binding

indica el tipo de transporte (http) y comunicación (Document, RPC - literal, encoded)



```
▼<binding name="PublicadorPortBinding" type="tns:Publicador">  
  <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="rpc"/>  
  ...  
</binding>
```

- la ubicación física del WS

```
  <binding>  
    ...  
  </binding>  
<service>  
  <service name="PublicadorService">  
    ▼<port name="PublicadorPort" binding="tns:PublicadorPortBinding">  
      <soap:address location="http://localhost:9128/publicador"/>  
    </port>  
  </service>
```

message constructs define abstract messages

message

Web Services

- Web Services Description Language (WSDL)

```
<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-
xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy" xmlns:wsam="http
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://schemas.xmlsoap.
  <types/>
  <message name="suma">
    <part name="arg0" type="xsd:int"/>
    <part name="arg1" type="xsd:int"/>
  </message>
  <message name="sumaResponse">
    <part name="return" type="xsd:int"/>
  </message>
  <portType name="Publicador">
    <operation name="suma" parameterOrder="arg0 arg1">
      <input wsam:Action="http://servidor/Publicador/sumaRequest" message="tns:suma"/>
      <output wsam:Action="http://servidor/Publicador/sumaResponse" message="tns:sumaResponse"/>
    </operation>
  </portType>
  <binding name="PublicadorPortBinding" type="tns:Publicador">
    <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="rpc"/>
    <operation name="suma">
      <soap:operation soapAction=""/>
      <input>
        <soap:body use="literal" namespace="http://servidor/">
      </input>
      <output>
        <soap:body use="literal" namespace="http://servidor/">
      </output>
    </operation>
  </binding>
  <service name="PublicadorService">
    <port name="PublicadorPort" binding="tns:PublicadorPortBinding">
      <soap:address location="http://localhost:9128/publicador"/>
    </port>
  </service>
</definitions>
```

tipos

Web Services

- Web Services Description Language (WSDL)

tipos

```
<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-
xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy" xmlns:wsam="http
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://schemas.xmlsoap.
  <types>
    <xsd:schema>
      <xsd:import namespace="http://publicar.complejoservidor/" schemaLocation="http://localhost:9128/publicador?xsd=1"/>
    </xsd:schema>
  </types>
  <message name="sumaResponse">
    <part name="return" type="xsd:int"/>
  </message>
  <portType name="Publicador">
    <operation name="suma" parameterOrder="arg0 arg1">
      <input wsam:Action="http://servidor/Publicador/sumaRequest" message="tns:suma"/>
      <output wsam:Action="http://servidor/Publicador/sumaResponse" message="tns:sumaResponse"/>
    </operation>
  </portType>
  <binding name="PublicadorPortBinding" type="tns:Publicador">
    <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="rpc"/>
    <operation name="suma">
      <soap:operation soapAction=""/>
      <input>
        <soap:body use="literal" namespace="http://servidor/">
      </input>
      <output>
        <soap:body use="literal" namespace="http://servidor/">
      </output>
    </operation>
  </binding>
  <service name="PublicadorService">
    <port name="PublicadorPort" binding="tns:PublicadorPortBinding">
      <soap:address location="http://localhost:9128/publicador"/>
    </port>
  </service>
</definitions>
```

Web Services

- Web Services Description Language (WSDL)

```
<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-
xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy" xmlns:wsam="http
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://schemas.xmlsoap.
```

```
<types/>
<message name="suma">
  <part name="arg0" type="xsd:int"/>
  <part name="arg1" type="xsd:int"/>
</message>
<message name="sumaResponse">
  <part name="return" type="xsd:int"/>
</message>
<portType name="Publicador">
  <operation name="suma" parameterOrder="arg0 arg1">
    <input wsam:Action="http://servidor/Publicador/sumaRequest" message="sumaRequest" type="xsd:int"/>
    <output wsam:Action="http://servidor/Publicador/sumaResponse" message="sumaResponse" type="xsd:int"/>
  </operation>
</portType>
<binding name="PublicadorPortBinding" type="tns:Publicador">
  <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="rpc"/>
  <operation name="suma">
    <soap:operation soapAction="http://servidor/Publicador/sumaRequest"/>
    <input>
      <soap:body use="literal" namespace="http://servidor/">
    </input>
    <output>
      <soap:body use="literal" namespace="http://servidor/">
    </output>
  </operation>
</binding>
<service name="PublicadorService">
  <port name="PublicadorPort" binding="tns:PublicadorPortBinding">
    <soap:address location="http://localhost:9128/publicador/">
  </port>
</service>
</definitions>
```

tipos

mensajes

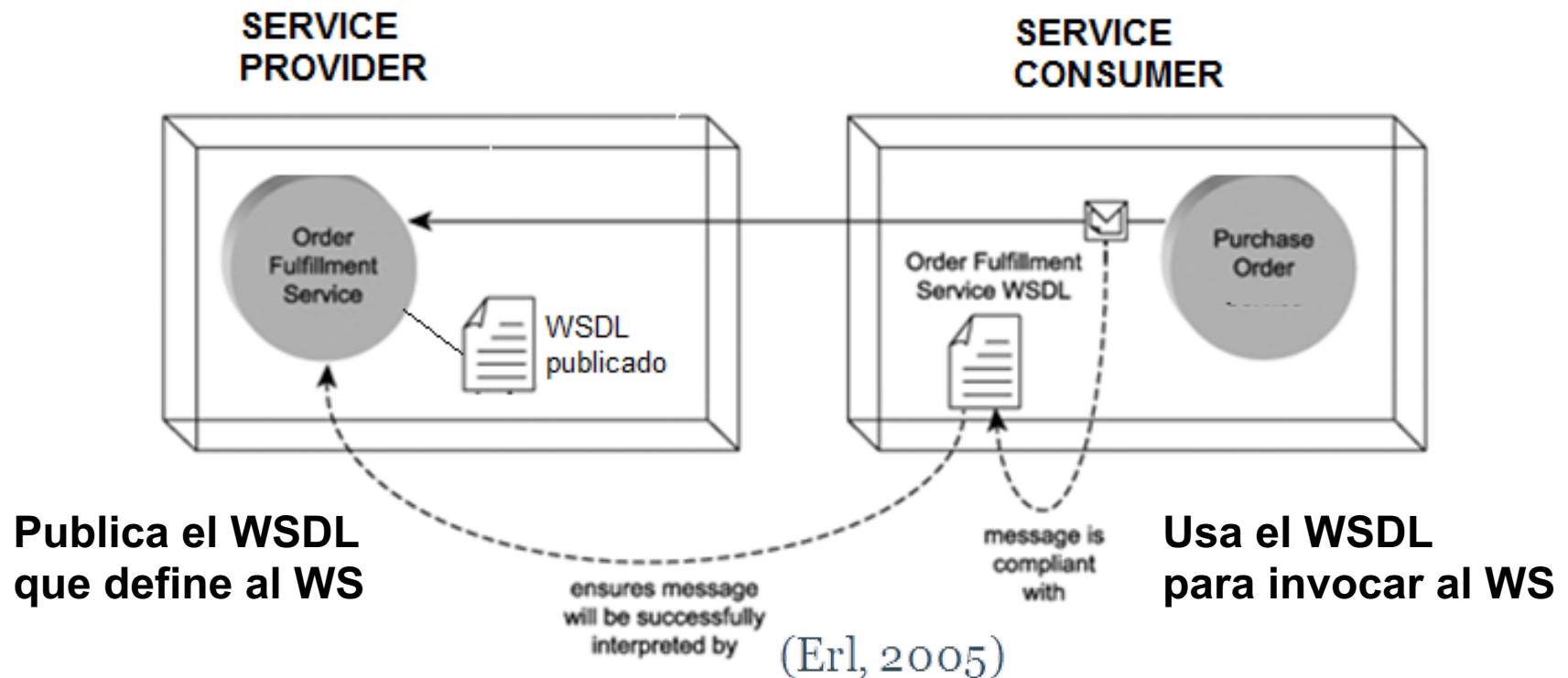
operaciones

Transporte y
comunicación

Ubicación física

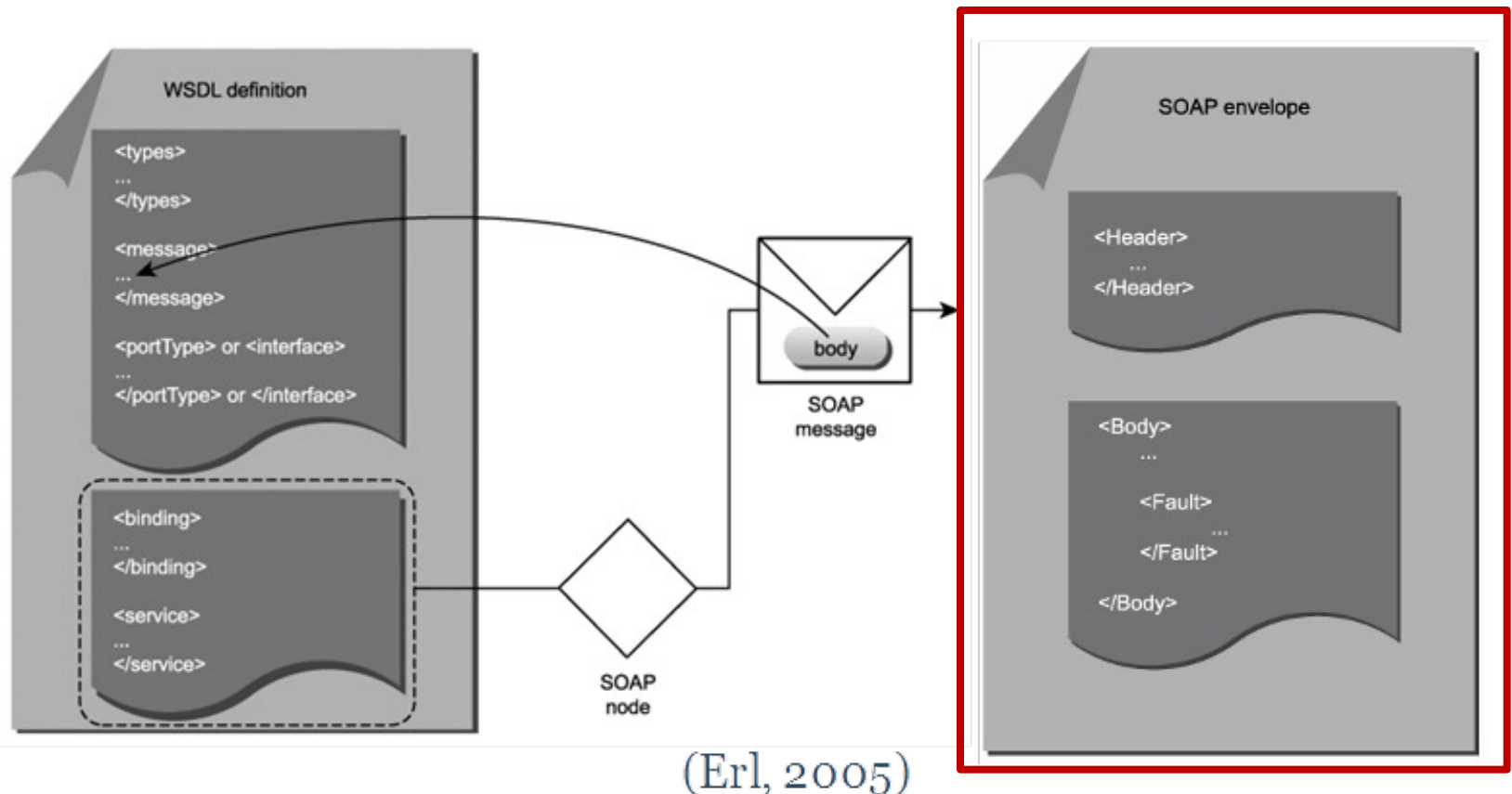
Web Services

- WS definidos desde el punto de vista del proveedor
 - funcionamiento con roles: **proveedor, consumidor**
 - podría haber un registro intermedio para descubrir WS



Web Services

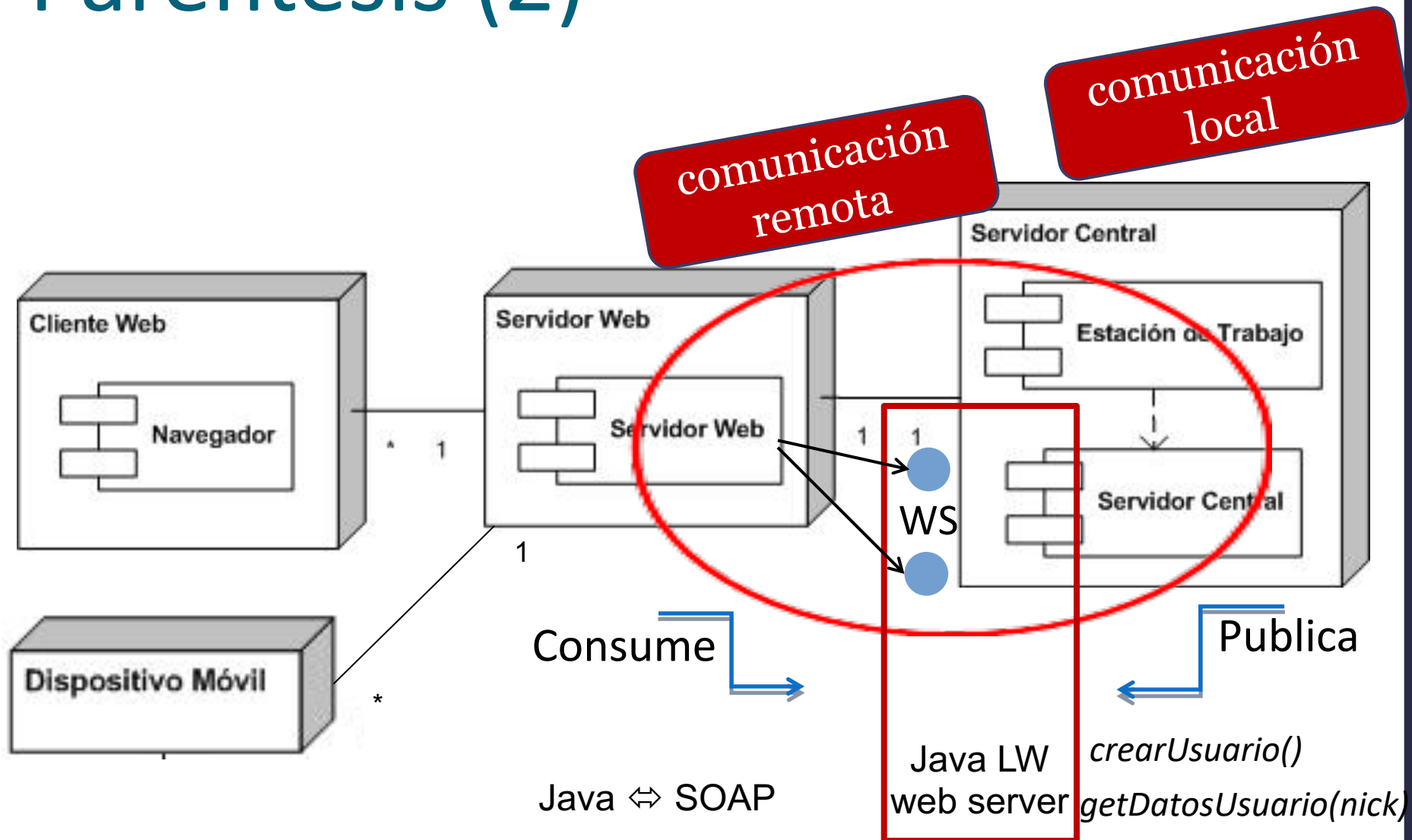
- Simple Object Access Protocol (SOAP)
 - protocolo estándar para intercambiar datos XML en WS



Paréntesis

- Hagamos un mapeo a nuestra realidad...
 - Servidor Web quiere consumir una Lógica (Servidor Central) que ya no es alcanzable de manera local
 - WS para lograr comunicarlos de manera remota
 - Servidor Central publica sus operaciones (en el Java Lightweight Web server), y el Servidor Web las consume.
 - Nuestro Servidor Web es el Cliente (consumidor).

Paréntesis (2)

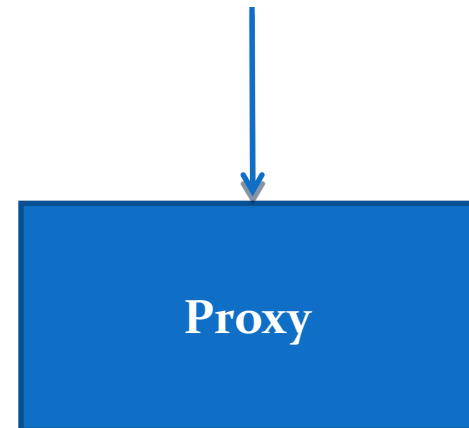
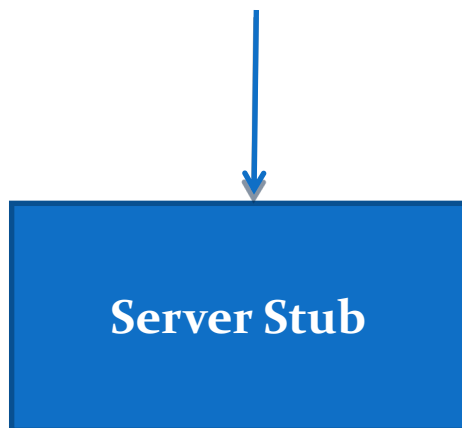


Web Services

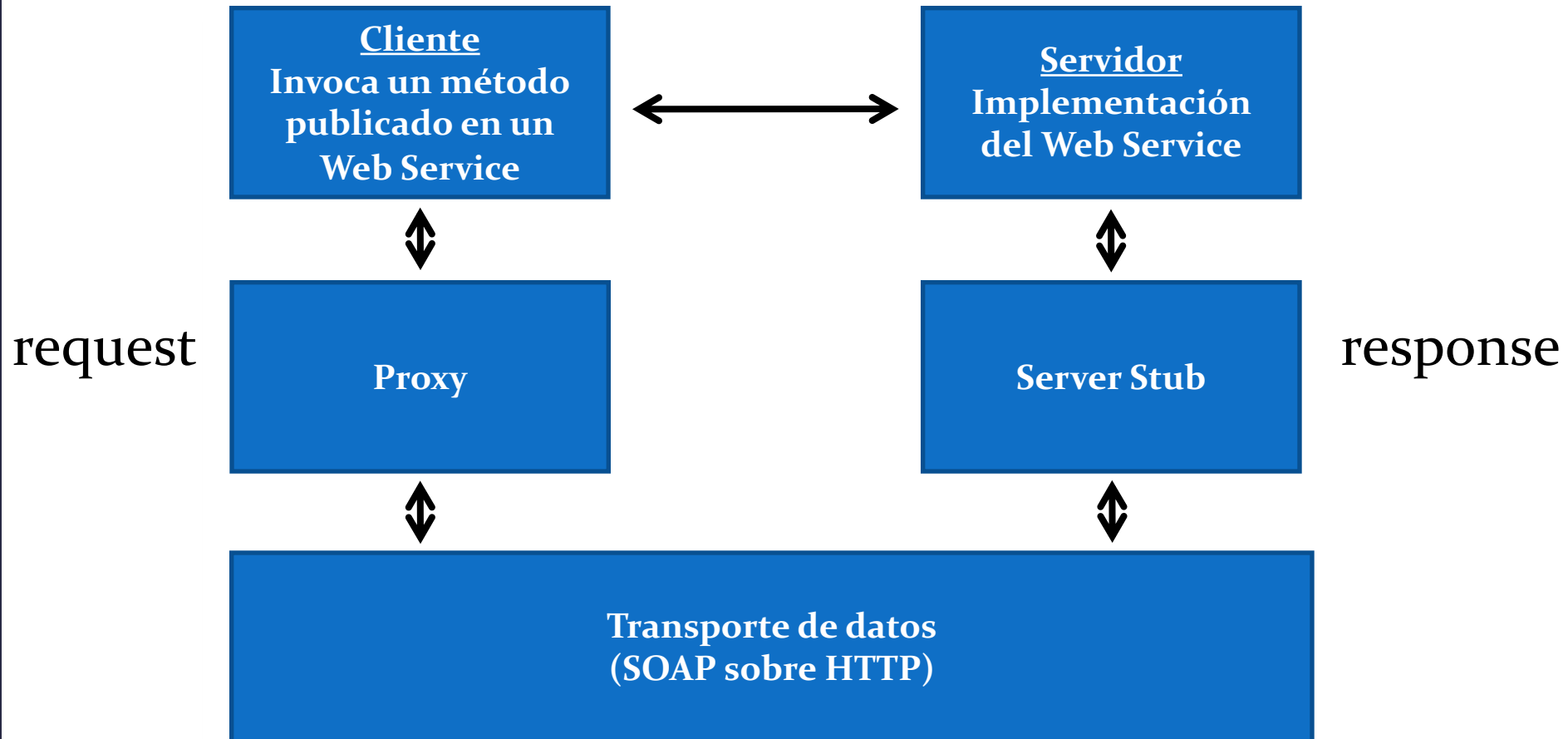
- Cliente (consumidor)
 - Es la entidad que desea consumir un servicio
- Servidor (proveedor)
 - Es la entidad que brinda la infraestructura para publicar un servicio y consumirlo
- A tener en cuenta...
 - El servidor tiene que publicar sus operaciones en un **lugar alcanzable**, y el cliente tiene que **saber el lugar exacto** para encontrar las operaciones.

Web Services

- Entonces
- el Servidor tiene que publicar sus funcionalidades, pero **¿Cómo lo hace?**
- el Cliente tiene que encontrar las operaciones publicadas para poder utilizarlas, **¿cómo hacerlo?**



Web Services



Web Services

- **Server Stub**

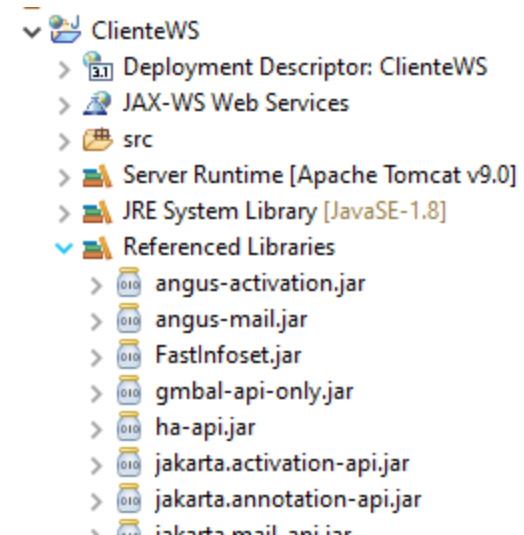
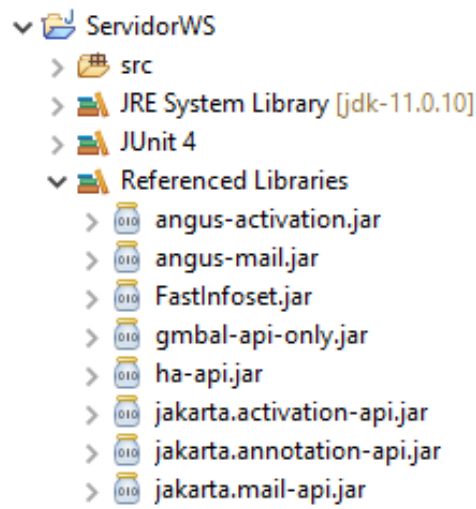
- Es quien representa la lógica del servicio
- Quien hace posible la publicación
- Existe en el entorno del servidor

- **Proxy**

- Es quien le brinda al cliente facilidades para acceder a un servicio.
- Encapsula la implementación asociada a dicho consumo.
- Construye objetos Java obtenidos del mensaje de respuesta, para ser utilizados “cómo si estuvieran locales” a la aplicación

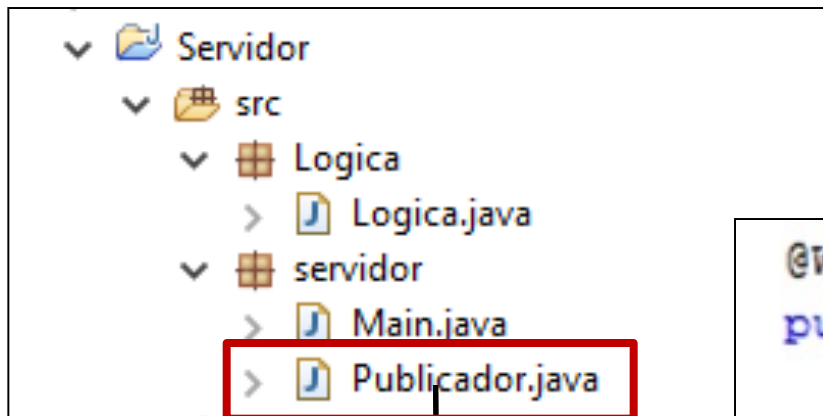
Jakarta XML Web Services

- Implementación de eclipse de XML Web Services
- En la siguiente página se puede descargar y encontrar documentación:
 - <https://eclipse-ee4j.github.io/metro-jax-ws/>
- Se debe incluir en el proyecto servidor (para publicar) y en el proyecto cliente (para consumir)



Web Services - Ejemplo 1

Servidor



Clase encargada de la publicación.

Método que quiero publicar

```
@WebMethod //Operación a publicar
public int suma(int i, int j){
    Logica log = new Logica();
    int ret = log.sumar(i,j);
    return ret;
}
```


Web Services - Ejemplo 1

- Publicando el servicio (Lightweight Web server)

```
@WebService //Indico que en esta clase hay operaciones a publicar
@SOAPBinding(style = Style.RPC, parameterStyle = ParameterStyle.WRAPPED)
public class Publicador {
    private Endpoint endpoint = null;
    //Constructor
    public Publicador(){}
    //Operaciones las cuales quiero publicar
    @WebMethod(exclude = true)
    public void publicar(){ //Operación que me genera el Server-Stub
        endpoint = Endpoint.publish("http://localhost:9128/publicador", this);
    }
    @WebMethod(exclude = true)
    public Endpoint getEndpoint() {
        return endpoint;
    }
    @WebMethod //Operación a publicar que consumiré en el Cliente
    public int suma(int i, int j){
        Logica log = new Logica();
        int ret = log.sumar(i,j);
        return ret;
    }
}
```

Lugar en donde
se publica el servicio



Web Services - Ejemplo 1

- Publicando el servicio (Lightweight Web server)

```
@WebService //Indico que en esta clase hay operaciones a publicar
@SOAPBinding(style = Style.RPC, parameterStyle = ParameterStyle.WRAPPED)
public class Publicador {
    private EndpointReference _endpoint;

    //Constructor
    public Publicador() {
        //Operaciones a publicar
        @WebMethod(excludeFromWSDL = true)
        public void publicar() {
            _endpoint = EndpointReference.create("http://localhost:8080/");
        }

        @WebMethod(excludeFromWSDL = true)
        public EndpointReference getEndpoint() {
            return _endpoint;
        }
    }

    @WebMethod //Operación a publicar que consume en el Cliente
    public int suma(int i, int j){
        Logica log = new Logica();
        int ret = log.sumar(i,j);
        return ret;
    }
}
```

Publicación del servicio (generación server stub)

Ar en donde
lico el servicio

Web Services - General

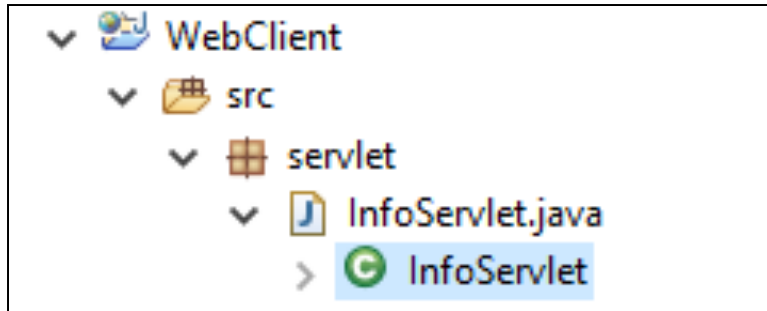
- Ejecuto la aplicación Servidor...
 - Esto realizará la publicación del servicio, registrando mi operación publicada en el lugar especificado.
 - Para verificar que se publicó correctamente, se puede ingresar en el browser: url-publicacion?wsdl
Ej. <http://localhost:9128/publicador?wsdl>
 - Al ingresar a la URL anterior, se visualiza el archivo XML en el browser. De esta manera me aseguro que la publicación ha sido con éxito
- Ahora es el turno del Cliente...

Web Services - General

- Para la aplicación cliente...
 - El cliente debe encontrar el servicio, indicándole el lugar exacto en donde debe consumir las operaciones.
 - Para la creación del Proxy se utilizará el comando:
 - `wsimport -keep wsdl_URL` (desde consola)
 - Este comando genera el código que permite al Cliente hacer uso del Servicio y crea una serie de objetos que son los que se utilizan de forma local.
 - Es imprescindible indicar la URL (lugar exacto) de donde se encuentran los métodos.

Web Services - Ejemplo 1

Cliente



Consumir un servicio

Ingresar:

Ingresar:

Llama

Lugar donde se
consumirá el servicio

```
<body>
  <h1>Consumir un servicio</h1>
  <form style="margin: -8.02px; width: 10px; height: auto;
    "action="infoServlet" ACTION="POST">
    <label>Ingresar:</label>
    <input id="nroi" type="text" name="nroi"/>
    <label>Ingresar:</label>
    <input id="nroj" type="text" name="nroj"/>
    <input type="submit" value="suma" name="operador" />
  </form>
</body>
```

Web Services - Ejemplo 1

Cliente – InfoServlet.java

```
// obtengo los valores de los input
```

```
String sumandoi = request.getParameter("nroi");
```

```
String sumandoj = request.getParameter("nroj");
```

```
int i = Integer.parseInt(sumandoi);
```

```
int j = Integer.parseInt(sumandoj);
```

```
int resultado = 0;
```

```
// consumo el servicio publicado
```

```
servidor.PublicadorService service = new servidor.PublicadorService();
```

```
servidor.Publicador port = service.getPublicadorPort();
```

```
resultado = port.suma(i, j);
```

```
// guardo el resultado como un atributo del request
```

```
request.setAttribute("res", resultado);
```

```
// redirijo a la jsp que muestra el resultado
```

```
RequestDispatcher dis = request.getRequestDispatcher("resultado.jsp");
```

```
dis.forward(request, response);
```

Web Services - Ejemplo 1

Cliente – InfoServlet.java

De esta manera se realiza una llamada al servicio publicado, en este caso a la operación *suma*.

Se utiliza el código generado, por El comando `-wsimport` o por el IDE

```
// obtengo los valores de los  
String sumandoi = request.get  
Parameter("nroj");  
ndoi);  
int j = Integer.parseInt(sumandoj);  
int resultado = 0;
```

```
// consumo el servicio publicado  
servidor.PublicadorService service = new servidor.PublicadorService();  
servidor.Publicador port = service.getPublicadorPort();  
resultado = port.suma(i, j);
```

```
// guardo el resultado como un atributo del request  
request.setAttribute("res", resultado);
```

```
// redirijo a la jsp que muestra el resultado  
RequestDispatcher dis = request.getRequestDispatcher("resultado.jsp");  
dis.forward(request, response);
```

Web Services - Resumen

- Pasos para crearlos
 - Definir componentes que comunican remotamente
 - Elegir que componente publica los servicios (Servidor)
 - Elegir que componente consume los servicios (Cliente)
 - Servidor
 - Anotar la clase (o interfaz) que correrá en el servidor
 - Definir la dirección en la que se publicará el servicio
 - Publicar y generar Stubs del servidor (WSDL del WS)
 - Cliente
 - Escribir un cliente que usa el servidor (ej. Servlet)
 - Generar los proxys necesarios desde el WSDL del WS
- Ejecutar el servidor y luego el cliente

Cuestiones prácticas

- Uso de anotaciones para la generación de WS y tipos válidos
- Annotations - Cuáles utilizar?
 - **@WebService**

A nivel de clase, indica que la misma debe ser expuesta como Web Service
 - **@WebMethod**

A nivel de método, indica que el método será incluido en la interfaz del servicio
 - **@WebParam**

A nivel de parámetro, indica el nombre y tipo que tomará dicho parámetro en el servicio.

Cuestiones prácticas (2)

- Annotations - Cuales utilizar? (cont.)
 - **@SOAPBinding**
Indica el estilo de codificación SOAP que se usará para el servicio (ej. RPC)
 - **@XmlAccessorType**
Determina el control sobre la serialización por defecto de las propiedades y atributos en una clase
- Tipos válidos
 - Nativos de java
 - Se mapean casi directamente con los nativos definidos en SOAP

Cuestiones prácticas (3)

- Tipos válidos (cont.)
 - Datatypes
 - Debe ser un Java Bean
 - Clase que cumple con ciertas convenciones
 - Entre ellas: Serializable, Constructor sin argumentos, métodos get y set
 - Excepciones
 - Se mapean a los SOAP Faults
 - También se serializan en XML
- Hay ciertos tipos de Java que no pueden ser publicados.
 - Ej. colecciones o tipos obsoletos (Vector, Date)

Web Services - Ejemplo 2

Datatype a publicar del lado del servidor

DataPersona

Determinamos el control sobre la serialización por defecto de las propiedades y atributos en una clase

```
@XmlAccessorType(XmlAccessType.FIELD)
public class DataMaestro {
    private String nombre;
    private String apellido;
    private ArrayList<DataPersona> convocados = new ArrayList<DataPersona>();
}
```

Web Services - Ejemplo 2

Métodos a publicar del lado del servidor

```
@WebMethod
public String obtenerApellido(DataPersona dp){
    Logica l = new Logica();
    return l.obtenerApellido(dp);
}

@WebMethod
public DataMaestro obtenerConvocados(String apellido){
    Logica l = new Logica();
    return l.obtenerConvocados(apellido);
}

@WebMethod
public byte[] getImage(@WebParam(name = "fileName") String name)
    throws IOException {
    byte[] byteArray = null;
    try {
        File f = new File(name);
        FileInputStream streamer = new FileInputStream(f);
        byteArray = new byte[streamer.available()];
        streamer.read(byteArray);
    } catch (IOException e) {
        throw e;
    }
    return byteArray;
}
```

Web Services - Ejemplo 2

Cliente – Reservados.java

Definimos el servicio para consumirlo.

```
complejoservidor.publicar.PublicadorService service =  
    new complejoservidor.publicar.PublicadorService();  
complejoservidor.publicar.Publicador port = service.getPublicadorPort();  
complejoservidor.publicar.DataMaestro maestro = port.obtenerConvocados("Tabárez");  
List<DataPersona> reservados = maestro.getConvocados();  
request.setAttribute("reservados", reservados);  
request.getRequestDispatcher("reservados.jsp").forward(request, response);
```

Cliente – reservados.jsp

Obtenemos el DataMaestro publicado para luego obtener la lista de convocados

```
<%  
    List<DataPersona> cont = (List)request.getAttribute("reservados");  
    Iterator<DataPersona> itr = cont.iterator();  
    int numero = 0;  
    while (itr.hasNext()) {  
        numero++;  
        DataPersona item = itr.next();  
    }  
%>
```

Mostramos los convocados obtenidos desde el request



DEMO